

able hardware accelerators and is able to find out your graphics hardware. Now if an application running on top of DirectFB issues a drawing call, say, `DrawRectangle()`, it is passed to the graphics accelerator, provided 'DrawRectangle' is supported in the chipset driver.

We will not go into more details on implementing the chipset driver (or `gfxdriver`) here as it is out of scope for this article.

Custom interface between DirectFB and hardware. It is possible to have your own software stack between DirectFB and the hardware. Fig. 3 shows a typical scenario. There are three layers in-between the graphics hardware and DirectFB in this scenario: porting layer, graphics driver wrapper and graphics driver.

Porting layer. Porting layer is a `gfxdriver` to hook your hardware with the DirectFB graphics core. It should implement standard DirectFB graphics functions to pass DirectFB calls (say, `DrawRectangle()`) to the underlying layer.

Graphics driver wrapper. This layer provides a wrapper between the hardware-specific graphics driver and DirectFB-specific `gfxdriver`. Like a typical wrapper, it provides modularity. If you need to use something other than DirectFB as the multimedia library (say, OpenGL), or the underlying graphics card is changed, the modularity will provide software reusability.

Graphics driver. The graphics driver can be a Linux char driver which can implement input/output controls (IOCTLs) to pass drawing calls from the wrapper to the hardware. It needs to implement 'mmap' to map the memory needed for windows and layers implemented by DirectFB. In a nutshell, typically, the graphics driver implements

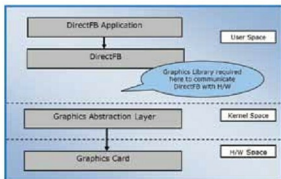


Fig. 1: Need of graphics library

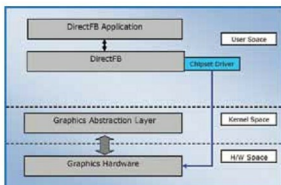


Fig. 2: Chipset driver

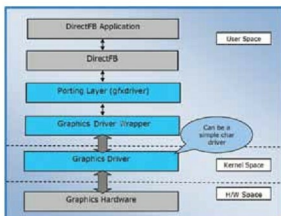


Fig. 3: Custom interface between DirectFB and hardware

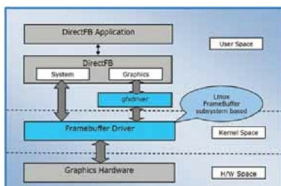


Fig. 4: Framebuffer driver between DirectFB and hardware

file operations such as `init`, `exit`, `open`, `close`, `mmap`, `munmap` and `ioctl`.

Note that in this scenario (and in the following scenario as well), the `gfxdriver` will not implement `dfb_gfxcard_map/unmap_mmap` and hardware-specific read/write because hardware access will be handled by the kernel-space driver.

Framebuffer driver between DirectFB and hardware. When DirectFB is configured to use 'fbdev' system (using DirectFB configuration option '`-enable-fbdev`'), DirectFB by default uses '`/dev/fb*`' interface for graphics acceleration. In this scenario, the graphics software stack is as shown in Fig. 4.

The software layers that the user has to develop are `gfxdriver` and framebuffer driver.

Gfxdriver. This is again the hook between the hardware and the DirectFB graphics core. It should implement standard DirectFB graphics functions to pass DirectFB calls to the framebuffer driver. The framebuffer driver can have IOCTLs for drawing functionalities; for example, IOCTL to draw a rectangle. The `gfxdriver` is responsible to pass such API calls (`DrawRectangle`) to framebuffer-driver IOCTL.

Framebuffer driver. Here a framebuffer driver is a Linux framebuffer sub-system based driver which is responsible for accessing the graphics hardware. The system core part of DirectFB can use the '`/dev/fb*`' interface directly. While the graphics core part of DirectFB needs the `gfxdriver` hook to access drawing-/blitting-specific IOCTLs implemented in the framebuffer driver. If any drawing/blitting IOCTL is not implemented, DirectFB uses software rendering. ●

The author is an embedded engineer at *efnlochs*